

8-Bit 8:1 Multiplexer: A Complete RTL to GDSII Implementation

Introduction

A multiplexer (Mux) is a combinational digital circuit that selects one of several input signals and forwards the selected input into a single output line. A multiplexer is often referred to as a data selector. An 8:1 multiplexer, specifically, has eight data input lines (D0 to D7), three select lines (S2, S1, S0) to determine which input is routed to the output, and a single output line (Y).

This project implements an 8-bit 8:1 multiplexer. This means that for each of the eight data inputs, there are 8 bits, resulting in a total of 64 data input lines (D0[7:0] to D7[7:0]) and an 8-bit output (Y[7:0]). Additionally, the design incorporates an active-low RESET signal to asynchronously set the output to a known state (typically all zeros) and an ENABLE signal to control the overall operation of the multiplexer. When ENABLE is high, the multiplexer operates normally; when ENABLE is low, the output is typically held at a high-impedance state or a fixed low state, regardless of the select or data inputs.

Operational Summary

Inputs	Description
Data Inputs	D0[7:0] to D7[7:0]: Eight sets of 8-bit data inputs.
Select Lines	S2, S1, S0: Select which input is passed to the output.
ENABLE	Active-high signal. When low, output is disabled.
RESET	Active-low asynchronous reset that sets output to 0.
Output	Y[7:0]: The selected 8-bit data output.

Truth Table (Simplified for 1-bit)

RESET	ENABLE	S2	S1	S0	Output (Y)
0	X	X	X	X	0 (Reset)
1	0	X	X	X	Z or 0

					(Disabled)
1	1	0	0	0	D0
1	1	0	0	1	D1
1	1	0	1	0	D2
1	1	0	1	1	D3
1	1	1	0	0	D4
1	1	1	0	1	D5
1	1	1	1	0	D6
RESET	ENABLE	S2	S1	S0	Output (Y)
0	X	X	X	X	0 (Reset)
1	0	X	X	X	Z or 0 (Disabled)
1	1	0	0	0	D0
1	1	0	0	1	D1
1	1	0	1	0	D2
1	1	0	1	1	D3
1	1	1	0	0	D4
1	1	1	0	1	D5
1	1	1	1	0	D6
1	1	1	1	1	D7

Applications

Multiplexers are fundamental components in digital design and have numerous applications, including:

- **Data Selection:** Routing multiple data streams through a single channel.
- **Data Routing:** Directing data from a single source to multiple destinations.
- **Parallel-to-Serial Conversion:** Converting parallel data into a serial stream.
- **Boolean Function Generation:** Implementing complex Boolean functions.
- **Memory Addressing:** Selecting specific memory locations.
- **Communication Systems:** Routing signals in telecommunication networks.
- **Test Equipment:** Selecting signals for measurement and analysis.

Tools Used

Design & Simulation

Tool	Purpose
Verilog HDL	Hardware Description Language used to design the RTL of the 8:1 Mux
Synopsys VCS	Compiles and simulates Verilog code at RTL level
Verdi	Waveform viewer for debugging

	simulation results and post-layout analysis
--	---

Synthesis

Tool	Purpose
Synopsys Design Compiler (dc_shell / dc_shell-xg-t)	Synthesizes RTL to gate-level netlist
Standard Design Cell Library (e.g., SAED32/45nm)	Target technology library for synthesis

Physical Design

Tool	Purpose
Synopsys ICC2 (icc2_shell)	Performs floorplanning, placement, CTS, routing, and GDSII generation
start_gui	Graphical interface for ICC2 and Design Compiler

Timing and Power Analysis

Tool	Purpose
Synopsys PrimeTime (pt_shell)	Static timing and power analysis after synthesis and layout

Environment

Tool	Purpose
Rocky Linux	Operating system environment used for running Synopsys tools

Project Structure

The project directory is organized as follows:

```
RTL2GDSII_/
```

```
├── 1_RTL/
```

```
| | └── 8x1mux.v
| | └── testbench.v
| └── 2_Synthesis/
| | └── 8x1mux_synth.v
| | └── synthesis_report.txt
| └── 3_Floorplan/
| | └── floorplan.tcl
| | └── floorplan_report.txt
| └── 4_PowerPlan/
| | └── power_plan.tcl
| | └── power_plan_report.txt
| └── 5_Place/
| | └── placement.tcl
| | └── placement_report.txt
| └── 6_CTS/
| | └── cts.cl
| | └── cts_report.txt
| └── 7_Routing/
| | └── routing.tcl
| | └── routing_report.txt
```

Below are the major steps followed using Synopsys tools:

1. RTL Simulation using VCS and waveform viewing in Verdi

In this step, the Register Transfer Level (RTL) design is validated by running simulations. Synopsys VCS compiles the Verilog code, and Verdi is used to visualize the simulation waveform which helps in debugging the logical behavior and timing of the design.

RTL code is translated to a gate-level representation using Synopsys Design Compiler. This involves logic optimization, mapping to standard cells, and applying design constraints such as timing and area.

Physical implementation includes several sequential stages:

- Floorplanning: Define chip size and block locations.
- Power Planning: Design the power distribution network.
- Placement: Allocate physical locations for logic cells.
- Clock Tree Synthesis (CTS): Build the clock distribution network with minimal skew.
- Routing: Establish metal interconnections for the placed cells.

Static Timing Analysis (STA) checks for setup and hold violations without requiring test vectors. Power analysis calculates dynamic and static power consumption. PrimeTime is used after layout to ensure the design meets timing and power budgets.

2. Synthesis using Design Compiler
3. Physical Design in ICC2 (Floorplan, Powerplan, Placement, CTS, Routing)
4. Static Timing Analysis and Power using PrimeTime

CODE & SNAPSHOTS

Create Verilog Files

File: mux_rtl.v

```
module mux (  
    input [7:0] I0, I1, I2, I3, I4, I5, I6, I7,  
    input [2:0] Sel,  
    input Clock,  
    input Reset, // Reset signal  
    input Enable, // Enable signal  
    output reg [7:0] Y;  
  
    always @ (posedge Clock or posedge Reset) begin  
        if (Reset) begin  
            Y <= 8'b0; // Reset output to 0  
        end else if (Enable) begin  
            case (Sel)  
                3'b000: Y <= I0;  
                3'b001: Y <= I1;  
                3'b010: Y <= I2;  
                3'b011: Y <= I3;  
                3'b100: Y <= I4;  
                3'b101: Y <= I5;  
                3'b110: Y <= I6;  
                3'b111: Y <= I7;  
                default: Y <= 8'b0;  
            endcase  
        end  
    end  
end
```

```
endmodule
```

File: mux_tb.v

```
`timescale 1ns/1ns

`include "mux_rtl.v" // includes the module definition for the 8-to-1 mux

module testbench;

    reg [7:0] I0, I1, I2, I3, I4, I5, I6, I7; // 8-bit inputs

    reg [2:0] Sel; // 3-bit select input

    reg Clock; // Clock input

    reg Reset; // Reset input

    reg Enable; // Enable input

    wire [7:0] Y; // 8-bit output

    // Instantiate the module under test

    mux dut (.I0(I0), .I1(I1), .I2(I2), .I3(I3), .I4(I4), .I5(I5), .I6(I6), .I7(I7), .Sel(Sel),
    .Clock(Clock), .Reset(Reset), .Enable(Enable), .Y(Y));

    // Clock generation

    always #5 Clock = ~Clock; // Clock signal with a period of 10 ns

    initial begin

        $fsdbDumpvars(); // Tool specific command for waveform generation

        // Initialize inputs

        I0 = 8'b00000000; I1 = 8'b00000001; I2 = 8'b00000010; I3 = 8'b00000011;

        I4 = 8'b00000100; I5 = 8'b00000101; I6 = 8'b00000110; I7 = 8'b00000111;

        Sel = 3'b000; // Start with the first input

        Clock = 0; // Initialize clock

        Reset = 1; // Start with reset

        Enable = 0; // Disable initially
```

```
#10 Reset = 0; // Release reset

Enable = 1; // Enable the mux


// Apply test cases

#20 Sel = 3'b000; // Select I0

#10; // Wait for clock edge

$display("Sel = %b, Y = %b", Sel, Y);


#20 Sel = 3'b001; // Select I1

#10; // Wait for clock edge

$display("Sel = %b, Y = %b", Sel, Y);


#20 Sel = 3'b010; // Select I2

#10; // Wait for clock edge

$display("Sel = %b, Y = %b", Sel, Y);


#20 Sel = 3'b011; // Select I3

#10; // Wait for clock edge

$display("Sel = %b, Y = %b", Sel, Y);


#20 Sel = 3'b100; // Select I4

#10; // Wait for clock edge

$display("Sel = %b, Y = %b", Sel, Y);


#20 Sel = 3'b101; // Select I5
```

```
#10; // Wait for clock edge

$display("Sel = %b, Y = %b", Sel, Y);


#20 Sel = 3'b110; // Select I6

#10; // Wait for clock edge

$display("Sel = %b, Y = %b", Sel, Y);


#20 Sel = 3'b111; // Select I7

#10; // Wait for clock edge

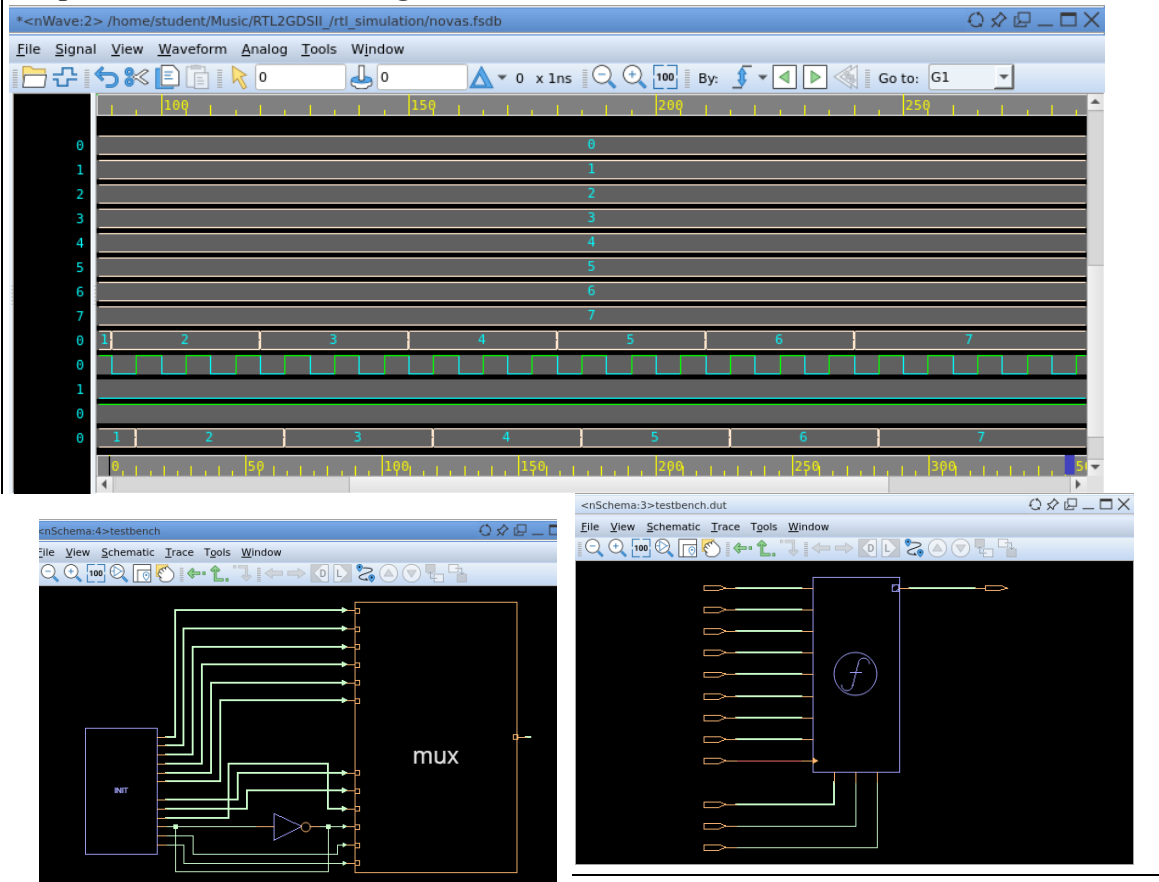
$display("Sel = %b, Y = %b", Sel, Y);


#100 $finish; // End simulation

end

endmodule
```


SnapShot of waveform and logic



DC SHELL

```
source -echo -verbose ./rm_setup/dc_setup.tcl
set RTL_SOURCE_FILES ../rtl/mux_rtl.v
define_design_lib WORK -path ./WORK

set_dont_use [get_lib_cells */FADD*]
set_dont_use [get_lib_cells */HADD*]
set_dont_use [get_lib_cells */AO*]
set_dont_use [get_lib_cells */OA*]
#set_dont_use [get_lib_cells */NAND*]
#set_dont_use [get_lib_cells */AND*]
set_dont_use [get_lib_cells */XOR*]
#set_dont_use [get_lib_cells */OR*]
set_dont_use [get_lib_cells */NOR*]
set_dont_use [get_lib_cells */XNOR*]
set_dont_use [get_lib_cells */MUX*]

analyze -format verilog ${RTL_SOURCE_FILES}
```

```
elaborate ${DESIGN_NAME}  
current_design
```

```
read_sdc ../../CONSTRAINTS/mux.sdc
```

```
#compile
```

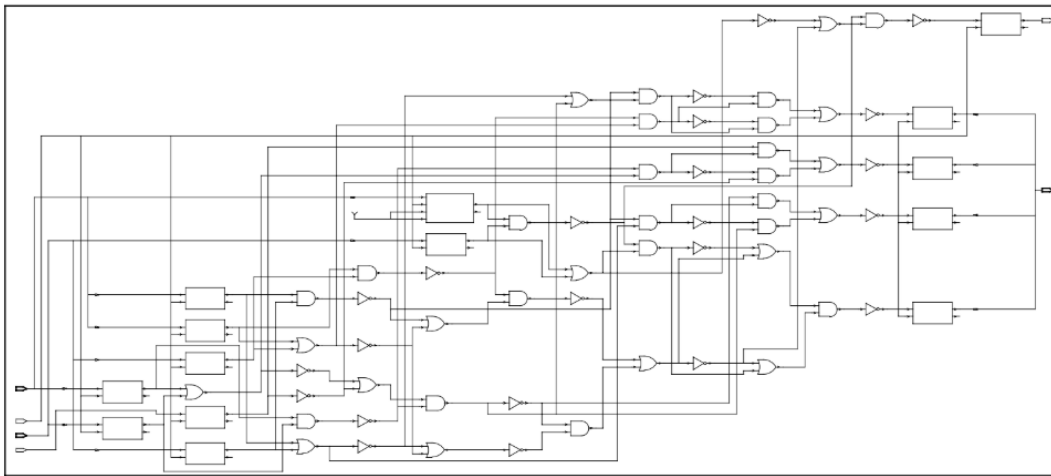
```
compile_ultra
```

```
#report_timing
```

```
write -format verilog -hierarchy -output
```

```
${RESULTS_DIR}/${DCRM_FINAL_VERILOG_OUTPUT_FILE}
```

Snapshot of Schematic of DC shell , report qor & report timing



Endpoint: SUM_reg[3] (rising edge-triggered flip-flop clocked by Clock)
 Path Group: Clock
 Path Type: max

Des/Clust/Port	Wire Load Model	Library	
mux	ForQA	saed32rvt_ss0p7vn40c	
Point	Incr	Path	
clock Clock (rise edge)	0.00	0.00	
clock network delay (ideal)	0.00	0.00	
reg2_reg[0]/CLK (DFFX1_RVT)	0.00	0.00 r	
reg2_reg[0]/Q (DFFX1_RVT)	0.71	0.71 r	
U11/Y (INVX1_RVT)	0.11	0.82 f	
U14/Y (OR2X1_RVT)	0.33	1.15 f	
U29/Y (AND2X1_RVT)	0.27	1.42 f	
U6/Y (NAND2X1_RVT)	0.38	1.81 r	
U17/Y (NAND3X0_RVT)	0.17	1.98 f	
U33/Y (NAND2X0_RVT)	0.24	2.22 r	
U13/Y (AND2X1_RVT)	0.31	2.53 r	
U4/Y (NAND2X1_RVT)	0.35	2.88 f	
U38/Y (AND2X1_RVT)	0.25	3.14 f	
SUM_reg[3]/D (DFFX1_RVT)	0.00	3.14 f	
data arrival time		3.14	
clock Clock (rise edge)	3.99	3.99	
clock network delay (ideal)	0.00	3.99	
clock uncertainty	-0.30	3.69	
SUM_reg[3]/CLK (DFFX1_RVT)	0.00	3.69 r	
library setup time	-0.53	3.16	
data required time		3.16	
data required time		3.16	
data arrival time		-3.14	
slack (MET)		0.02	

Report : qor
Design : mux
Version: W-2024.09
Date : Tue Jun 3 09:05:05 2025

Timing Path Group 'Clock'

Levels of Logic: 9.00
Critical Path Length: 3.14
Critical Path Slack: 0.02
Critical Path Clk Period: 3.99
Total Negative Slack: 0.00
No. of Violating Paths: 0.00
Worst Hold Violation: 0.00
Total Hold Violation: 0.00
No. of Hold Violations: 0.00

Cell Count

Hierarchical Cell Count: 0
Hierarchical Port Count: 0
Leaf Cell Count: 61
Buf/Inv Cell Count: 7
Buf Cell Count: 0
Inv Cell Count: 7
CT Buf/Inv Cell Count: 0
Combinational Cell Count: 47
Sequential Cell Count: 14
Macro Count: 0

Area

Combinational Area: 90.221121
Noncombinational Area: 92.508419
Buf/Inv Area: 8.895040
Total Buffer Area: 0.00
Total Inverter Area: 8.90
Macro/Black Box Area: 0.000000
Net Area: 11.740340

Cell Area: 182.729540
Design Area: 194.469881

Design Rules

Total Number of Nets: 74
Nets With Violations: 10
Max Trans Violations: 10
Max Cap Violations: 0

Hostname: ict-chipin.sot.pdpu.ac.in

Compile CPU Statistics

Resource Sharing: 0.01
Logic Optimization: 0.39
Mapping Optimization: 0.36

Overall Compile Time: 3.18
Overall Compile Wall Clock Time: 3.58

ICC2 SHELL

Floorplan

```

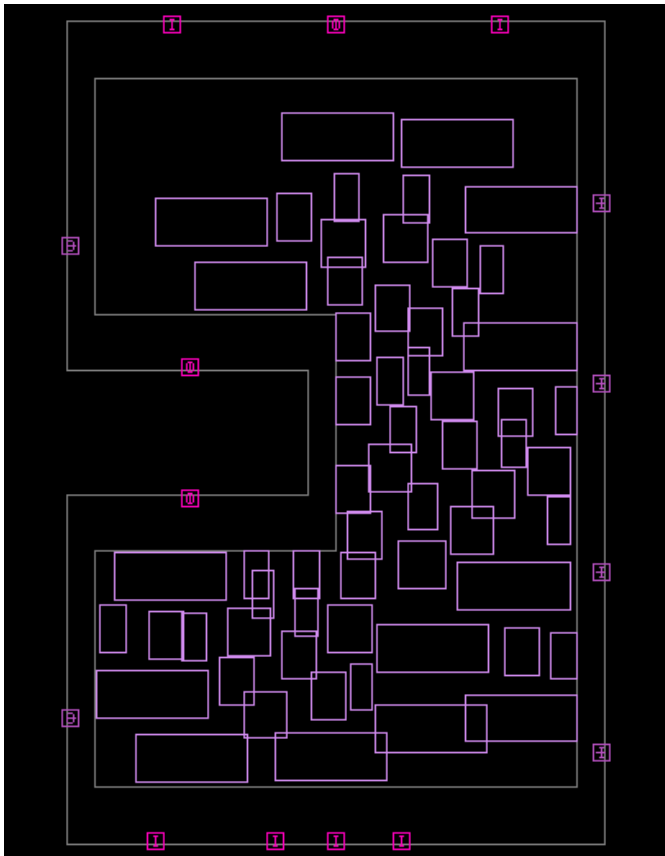
set PDK_PATH ../../ref

create_lib -ref_lib $PDK_PATH/lib/ndm/saed32rvt_c.ndm MUX_LIB

read_verilog {../../DC/results/mux.mapped.v} -library MUX_LIB -design mux -top mux

open_lib MUX_LIB
open_block MUX
initialize_floorplan -core_utilization 0.5 -coincident_boundary true -core_offset {1 2} -shape
U -orientation W
place_pins -self
create_placement -floorplan
save_block -as MUX_
save_lib

```



```

Area
-----
Combinational Area:          90.22
Noncombinational Area:      92.51
Buf/Inv Area:                8.90
Total Buffer Area:          0.00
Total Inverter Area:        8.90
Macro/Black Box Area:       0.00
Net Area:                    0
Net XLength:                 249.09
Net YLength:                 242.09
-----
Cell Area (netlist):          182.73
Cell Area (netlist and physical only): 182.73
Net Length:                   491.18

Design Rules
-----
Total Number of Nets:        75
Nets with Violations:         0
Max Trans Violations:         0
Max Cap Violations:           0
-----

```

```

Cell Count
-----
Hierarchical Cell Count:          0
Hierarchical Port Count:          0
Leaf Cell Count:                  61
Buf/Inv Cell Count:               7
Buf Cell Count:                   0
Inv Cell Count:                   7
Combinational Cell Count:         47
  Single-bit Isolation Cell Count: 0
  Multi-bit Isolation Cell Count:  0
  Isolation Cell Banking Ratio:    0.00%
  Single-bit Level Shifter Cell Count: 0
  Multi-bit Level Shifter Cell Count: 0
  Level Shifter Cell Banking Ratio: 0.00%
  Single-bit ELS Cell Count:        0
  Multi-bit ELS Cell Count:         0
  ELS Cell Banking Ratio:           0.00%
Sequential Cell Count:            14
  Integrated Clock-Gating Cell Count: 0
  Sequential Macro Cell Count:       0
  Single-bit Sequential Cell Count:  14
  Multi-bit Sequential Cell Count:   0
  Sequential Cell Banking Ratio:     0.00%
  BitsPerflop:                      1.00
Macro Count:                      0
-----

Area
-----
Combinational Area:               90.22
Noncombinational Area:            92.51
Buf/Inv Area:                     8.90
Total Buffer Area:                 0.00
Total Inverter Area:              8.90

```

PowerPlan

```

### Step 1: to_create power/ground nets and to_connect power/ground nets :-

```

```

#to create power nets

```

```

create_net -power {VDD}
create_net -ground {VSS}

```

```

## to connect power/ground_nets
connect_pg_net -all_blocks -automatic

```

```

### step 2: to create power and ground ring patterns
#scenario1:

```

```

create_pg_ring_pattern core_ring_pattern -horizontal_layer M7 -horizontal_width .4 -
horizontal_spacing .3 -vertical_layer M8 -vertical_width .4 -vertical_spacing .3

```

```
set_pg_strategy core_power_ring -core -pattern {{name : core_ring_pattern} {nets : {VDD
VSS}} {offset : {.5 .5}}}
```

```
compile_pg -strategies core_power_ring
```

```
### step 3: to create pg mesh pattern
```

```
create_pg_mesh_pattern mesh -layers { {{vertical_layer: M6} {width: .34} {spacing:
interleaving} {pitch: 5} {offset: .5}} {{horizontal_layer: M7} {width: .38} {spacing:
interleaving} {pitch: 5} {offset: .5}} {{vertical_layer: M8} {width: .38} {spacing:
interleaving} {pitch: 5} {offset: .5}}}
```

```
set_pg_strategy core_mesh -pattern { {pattern:mesh} {nets: VDD VSS}} -core -extension
{stop: innermost_ring}
```

```
#set_pg_strategy core_mesh -pattern { {pattern:mesh} {nets: VDD VSS}} -core -extension
{{{side: 1 2} {direction: L T} {stop: innermost_ring}}}
```

```
compile_pg -strategies core_mesh
```

```
###step 4 : to create std cell power rail pattern
```

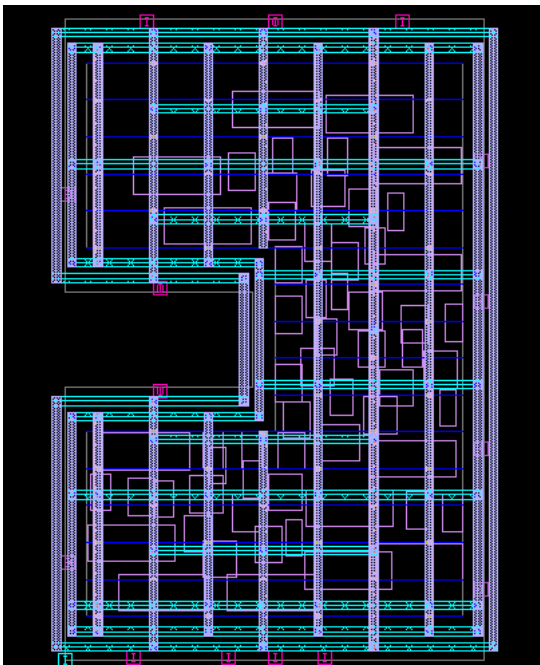
```
create_pg_std_cell_conn_pattern std_cell_rail -layers {M1} -rail_width 0.06
```

```
set_pg_strategy rail_strat -core -pattern {{name: std_cell_rail} {nets: VDD VSS} }
```

```
compile_pg -strategies rail_strat
connect_pg_net -all_blocks -automatic
#compile_pg
```

```
### step 5 : To save block and Save lib
```

```
save_block
save_lib
```



```
Cell Count
-----
Hierarchical Cell Count:      0
Hierarchical Port Count:      0
Leaf Cell Count:              61
Buf/Inv Cell Count:           7
Buf Cell Count:               0
Inv Cell Count:               7
Combinational Cell Count:     47
  Single-bit Isolation Cell Count: 0
  Multi-bit Isolation Cell Count: 0
  Isolation Cell Banking Ratio: 0.00%
  Single-bit Level Shifter Cell Count: 0
  Multi-bit Level Shifter Cell Count: 0
  Level Shifter Cell Banking Ratio: 0.00%
  Single-bit ELS Cell Count: 0
  Multi-bit ELS Cell Count: 0
  ELS Cell Banking Ratio: 0.00%
Sequential Cell Count:        14
  Integrated Clock-Gating Cell Count: 0
  Sequential Macro Cell Count: 0
  Single-bit Sequential Cell Count: 14
  Multi-bit Sequential Cell Count: 0
  Sequential Cell Banking Ratio: 0.00%
  BitsPerflop: 1.00
Macro Count: 0
-----
```

```

Area
-----
Combinational Area:          90.22
Noncombinational Area:      92.51
Buf/Inv Area:                8.90
Total Buffer Area:           0.00
Total Inverter Area:         8.90
Macro/Black Box Area:        0.00
Net Area:                    0
Net XLength:                 249.09
Net YLength:                 242.09
-----
Cell Area (netlist):          182.73
Cell Area (netlist and physical only): 182.73
Net Length:                   491.18

Design Rules
-----
Total Number of Nets:        77
Nets with Violations:         0
Max Trans Violations:         0
Max Cap Violations:           0
-----

```

Placement

```

set model "func"
set corner1 "nom"
set scenario1 "${model}::${corner1}"
remove_modes -all; remove_corners -all; remove_scenarios -all

create_mode $model
create_corner $corner1
create_scenario -name func::nom -mode func -corner nom
current_mode func
current_scenario func::nom

source ../../CONSTRAINTS/mux.sdc

set_dont_use [get_lib_cells */FADD*]
set_dont_use [get_lib_cells */HADD*]
set_dont_use [get_lib_cells */AO*]
set_dont_use [get_lib_cells */OA*]
#set_dont_use [get_lib_cells */NAND*]
#set_dont_use [get_lib_cells */AND*]
set_dont_use [get_lib_cells */XOR*]
#set_dont_use [get_lib_cells */OR*]
set_dont_use [get_lib_cells */NOR*]
set_dont_use [get_lib_cells */XNOR*]
set_dont_use [get_lib_cells */MUX*]
current_corner nom

```



```

current_scenario func::nom

set parasitic1 "p1"
set tluplus_file$parasitic1 "$PDK_PATH/tech/star_rcxt/saed32nm_1p9m_Cmax.tluplus"
set layer_map_file$parasitic1 "$PDK_PATH/tech/star_rcxt/saed32nm_tf_itf_tluplus.map"

set parasitic2 "p2"
set tluplus_file$parasitic2 "$PDK_PATH/tech/star_rcxt/saed32nm_1p9m_Cmin.tluplus"
set layer_map_file$parasitic2 "$PDK_PATH/tech/star_rcxt/saed32nm_tf_itf_tluplus.map"

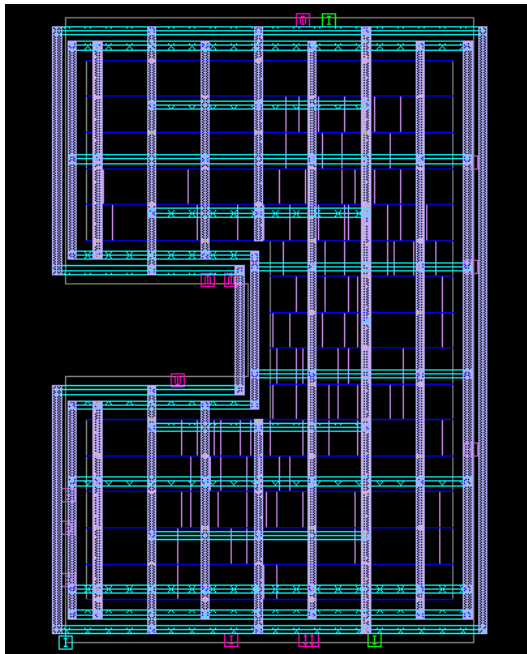
read_parasitic_tech -tlup $tluplus_filep1 -layermap $layer_map_filep1 -name p1
read_parasitic_tech -tlup $tluplus_filep2 -layermap $layer_map_filep2 -name p2

set_parasitic_parameters -late_spec $parasitic1 -early_spec $parasitic2
set_app_options -name place.coarse.continue_on_missing_scandef -value true

place_pins -self
place_opt
legalize_placement

save_block -as mux_placement
save_lib

```



```

Cell Count
-----
Hierarchical Cell Count:          0
Hierarchical Port Count:          0
Leaf Cell Count:                  59
Buf/Inv Cell Count:               8
Buf Cell Count:                   0
Inv Cell Count:                   8
Combinational Cell Count:         45
  Single-bit Isolation Cell Count: 0
  Multi-bit Isolation Cell Count:  0
  Isolation Cell Banking Ratio:    0.00%
  Single-bit Level Shifter Cell Count: 0
  Multi-bit Level Shifter Cell Count: 0
  Level Shifter Cell Banking Ratio: 0.00%
  Single-bit ELS Cell Count:        0
  Multi-bit ELS Cell Count:         0
  ELS Cell Banking Ratio:           0.00%
Sequential Cell Count:             14
  Integrated Clock-Gating Cell Count: 0
  Sequential Macro Cell Count:       0
  Single-bit Sequential Cell Count:  14
  Multi-bit Sequential Cell Count:   0
  Sequential Cell Banking Ratio:     0.00%
  BitsPerflop:                      1.00
Macro Count:                       0
-----

```

```
Area
-----
Combinational Area:          74.46
Noncombinational Area:      92.51
Buf/Inv Area:                10.17
Total Buffer Area:           0.00
Total Inverter Area:         10.17
Macro/Black Box Area:        0.00
Net Area:                     0
Net XLength:                  193.83
Net YLength:                  267.29
-----
Cell Area (netlist):          166.97
Cell Area (netlist and physical only): 166.97
Net Length:                   461.12

Design Rules
-----
Total Number of Nets:        74
Nets with Violations:         9
Max Trans Violations:         9
Max Cap Violations:           0
-----
```

CTS (Clock Tree Synthesis)

```
check_design -checks pre_clock_tree_stage
```

```
##stage 1:
```

```
synthesize_clock_tree
```

```
##stage 2:
```

```
set_app_options -name cts.optimize.enable_local_skew -value true
```

```
set_app_options -name cts.compile.enable_local_skew -value true
```

```
set_app_options -name cts.compile.enable_global_route -value false
```

```
set_app_options -name clock_opt.flow.enable_ccd -value true
```

```
clock_opt -to build_clock
```

```
#stage 3:
```

```
clock_opt -from route_clock -to route_clock
```

```
clock_opt
```

```
#report_clock_qor
#report_clock_qor -largest 2 -show_verbose_paths
#report_clock_qor -largest 2 -show_verbose_paths > cts_ccd.rpt

save_block -as mux_cts_CCD
save_lib

# set app options
set_app_options -name route.global.timing_driven -value true
set_app_options -name route.global.crosstalk_driven -value false

#track assignment
set_app_options -name route.track.timing_driven -value true
set_app_options -name route.track.crosstalk_driven -value true

#detail route
set_app_options -name route.detail.timing_driven -value true
set_app_options -name route.detail.save_after_iterations -value false
set_app_options -name route.detail.force_max_number_iterations -value false
set_app_options -name route.detail.antenna -value true
set_app_options -name route.detail.antenna_fixing_preference -value use_diodes
set_app_options -name route.detail.diode_libcell_names -value */ANTENNA_RVT

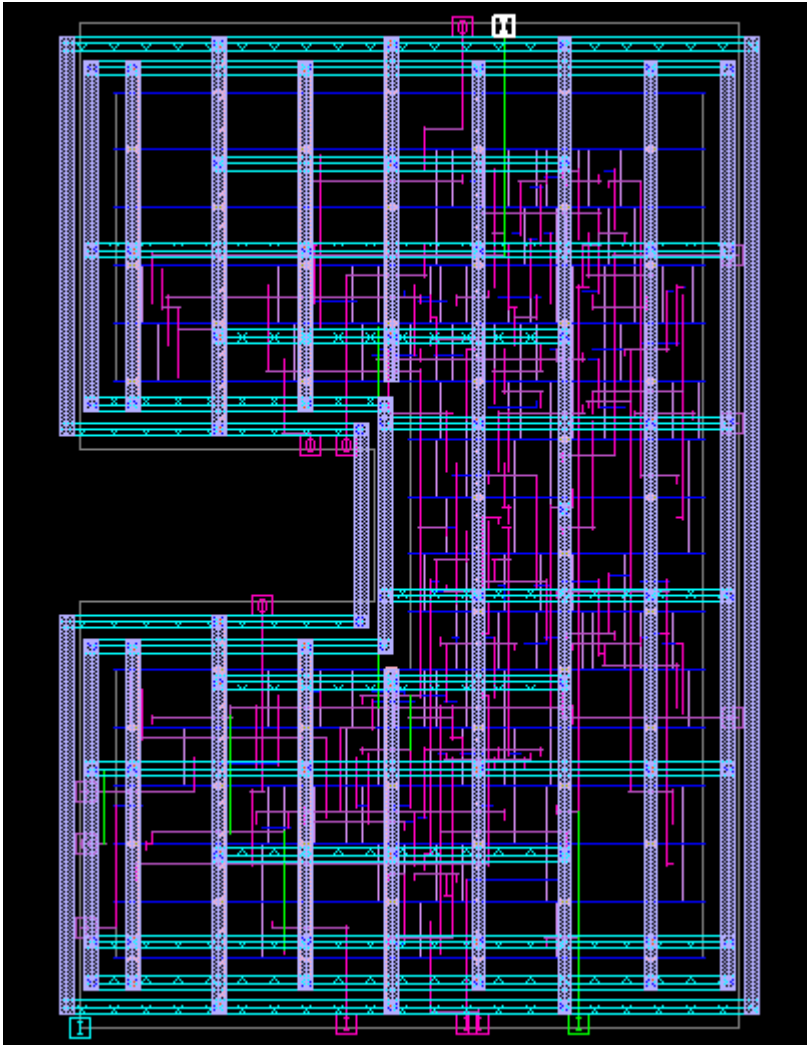
route_global
#save_block route_global_database

route_track
#save_block route_track_database

route_detail
#save_block route_detail_database

#route_auto
route_opt

write_verilog ./results/mux.routed.v
write_sdc -output ./results/mux.routed.sdc
write_parasitics -format spef -output ./results/mux_${scenario1}.spef
```



```

-----
Levels of Logic:                                0
Critical Path Length:                          0.50
Critical Path Slack:                           3.09
Critical Path Clk Period:                      3.99
Total Negative Slack:                          0.00
No. of Violating Paths:                        0
-----

```

```

Scenario          'func::nom'
Timing Path Group '**reg2out_default**'
-----

```

```

Levels of Logic:                                0
Critical Path Length:                          0.08
Critical Path Slack:                           3.07
Critical Path Clk Period:                      3.99
Total Negative Slack:                          0.00
No. of Violating Paths:                        0
-----

```

```

Scenario          'func::nom'
Timing Path Group 'Clock'
-----

```

```

Levels of Logic:                                11
Critical Path Length:                          0.35
Critical Path Slack:                           3.31
Critical Path Clk Period:                      3.99
Total Negative Slack:                          0.00
No. of Violating Paths:                        0
Worst Hold Violation:                          0.00
Total Hold Violation:                          0.00
No. of Hold Violations:                        0
-----

```

Cell Count

```

-----
Hierarchical Cell Count:      0
Hierarchical Port Count:     0
Leaf Cell Count:             75
Buf/Inv Cell Count:          24
Buf Cell Count:              2
Inv Cell Count:              22
Combinational Cell Count:    61
  Single-bit Isolation Cell Count:  0
  Multi-bit Isolation Cell Count:  0
  Isolation Cell Banking Ratio:    0.00%
  Single-bit Level Shifter Cell Count: 0
  Multi-bit Level Shifter Cell Count: 0
  Level Shifter Cell Banking Ratio: 0.00%
  Single-bit ELS Cell Count:        0
  Multi-bit ELS Cell Count:        0
  ELS Cell Banking Ratio:          0.00%
Sequential Cell Count:       14
  Integrated Clock-Gating Cell Count: 0
  Sequential Macro Cell Count:  0
  Single-bit Sequential Cell Count: 14
  Multi-bit Sequential Cell Count: 0
  Sequential Cell Banking Ratio: 0.00%
  BitsPerflop:                1.00
Macro Count:                  0
-----

```

Area

```

-----
Combinational Area:          90.22
Noncombinational Area:      92.51
Buf/Inv Area:                32.53
Total Buffer Area:           4.57
Total Inverter Area:         27.96
Macro/Black Box Area:        0.00
Net Area:                    0
Net XLength:                 223.60
Net YLength:                 313.19
-----
Cell Area (netlist):          182.73
Cell Area (netlist and physical only): 182.73
Net Length:                   536.79
-----

```

Design Rules

```

-----
Total Number of Nets:        90
Nets with Violations:        10
Max Trans Violations:        10
Max Cap Violations:          0
-----

```

Attributes

```

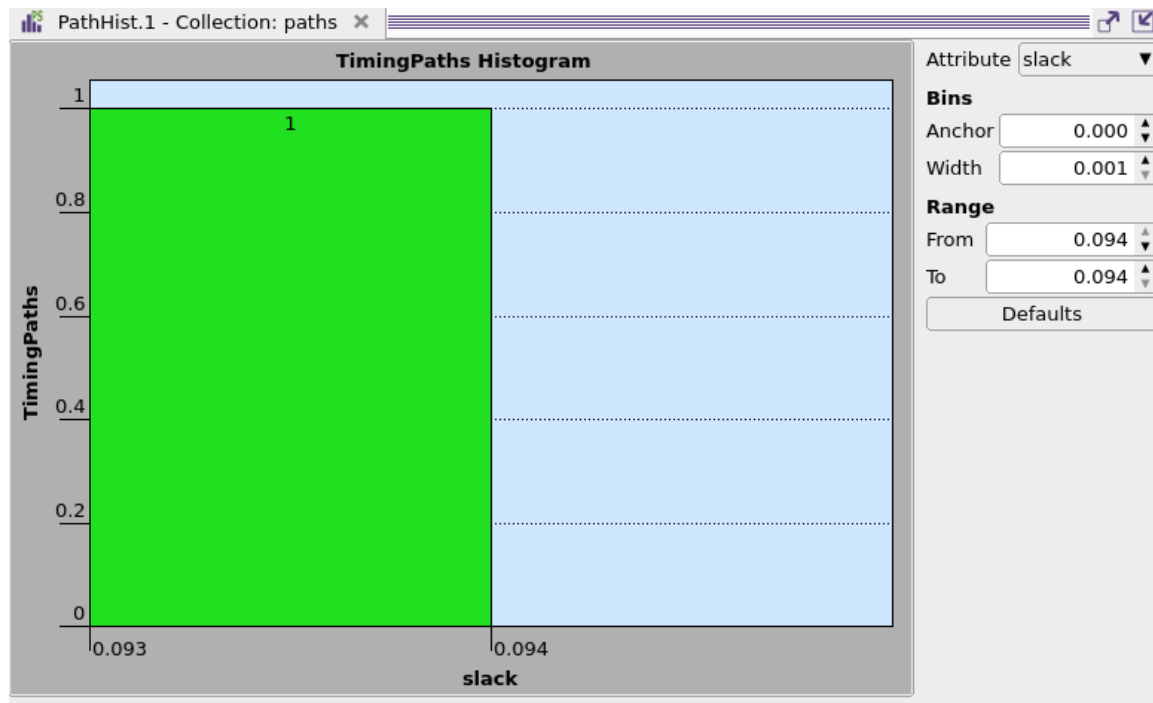
-----
u - User defined power group
i - Includes clock pin internal power

```

Power Group	Internal Power	Switching Power	Leakage Power	Total Power	(%)
Attrs					
io_pad	0.00e+00	0.00e+00	0.00e+00	0.00e+00	(0.0%)
memory	0.00e+00	0.00e+00	0.00e+00	0.00e+00	(0.0%)
black_box	0.00e+00	0.00e+00	0.00e+00	0.00e+00	(0.0%)
clock_network	2.43e+07	3.06e+06	2.44e+07	5.17e+07	(13.6%)
i_register	-1.46e+07	2.92e+05	2.86e+08	2.72e+08	(71.6%)
sequential	0.00e+00	0.00e+00	0.00e+00	0.00e+00	(0.0%)
combinational	1.05e+06	8.28e+05	5.40e+07	5.58e+07	(14.7%)
Total	1.08e+07 pW	4.18e+06 pW	3.64e+08 pW	3.79e+08 pW	

PT SHELL

The pt sleek time cam out to be 0.094 as it is shown in the graph below .



Final Summary

Conclusion: A Journey from RTL to GDSII – Learnings and Reflections

This project provided an invaluable, end-to-end experience in digital design—starting from the abstract world of RTL coding to the tangible physical layout ready for fabrication (GDSII). Implementing an 8-bit 8:1 multiplexer with RESET and ENABLE functionality was more than just a design task—it was an educational journey that deepened my understanding of both the theoretical and practical aspects of VLSI design.

At the RTL design stage, I strengthened my command over Verilog HDL, learning how to model functional behavior of digital circuits accurately and modularly. Writing a well-structured testbench and observing waveform outputs using VCS and Verdi taught me how simulation bridges the gap between code and logic.

During synthesis with Design Compiler, I realized the importance of optimization and constraint-driven development. The way high-level RTL code is translated into a gate-level netlist helped me appreciate the importance of timing, area, and power in real-world design.

Entering the physical design phase using ICC2, I learned about chip planning at the silicon level. Concepts like floorplanning, power distribution networks, standard cell placement, clock tree synthesis (CTS), and routing were no longer abstract terms—they became real challenges to address and resolve using specific commands, flows, and strategies.

The power planning steps made me more conscious of how crucial power integrity is, especially in deep-submicron technologies. Meanwhile, the placement and routing phases taught me about physical constraints and congestion handling. Clock Tree Synthesis particularly stood out as a critical balance between skew, latency, and power—a concept I had only briefly understood before this project.

Using PrimeTime for timing and power analysis, I saw firsthand how static analysis tools verify whether the design will function reliably across different operating conditions. Understanding concepts like slack, parasitic extraction, and path delays was key in realizing the real challenges of closing timing.

In short, this project taught me how various stages of the VLSI flow are intricately connected. Each step builds on the last, and mistakes or oversights in early phases often propagate downstream. This instilled in me a sense of discipline and carefulness while designing and scripting at each stage.

Most importantly, I gained practical exposure to industry-standard EDA tools from Synopsys—VCS, Design Compiler, ICC2, and PrimeTime—which are essential in modern semiconductor design workflows. I now feel more confident and prepared to handle larger and more complex ASIC design projects in the future

References

- <https://www.allaboutcircuits.com/>
- <https://www.vlsiexpert.com/>
- <https://www.chipgrad.com/>
- SAED 32nm PDK Documentation – Synopsys University Program
- Synopsys Tool User Guides
- VLSI Lab Tutorials by Mr. Puneet Mittal

Credits

This project was completed as part of the RTL to GDSII Flow Lab at Pandit Deendayal Energy University (PDEU).

We express our sincere gratitude to:

- Mr. Puneet Mittal, VLSI Expert, for his exceptional guidance
 - Faculty members of PDEU for their support and instruction
-